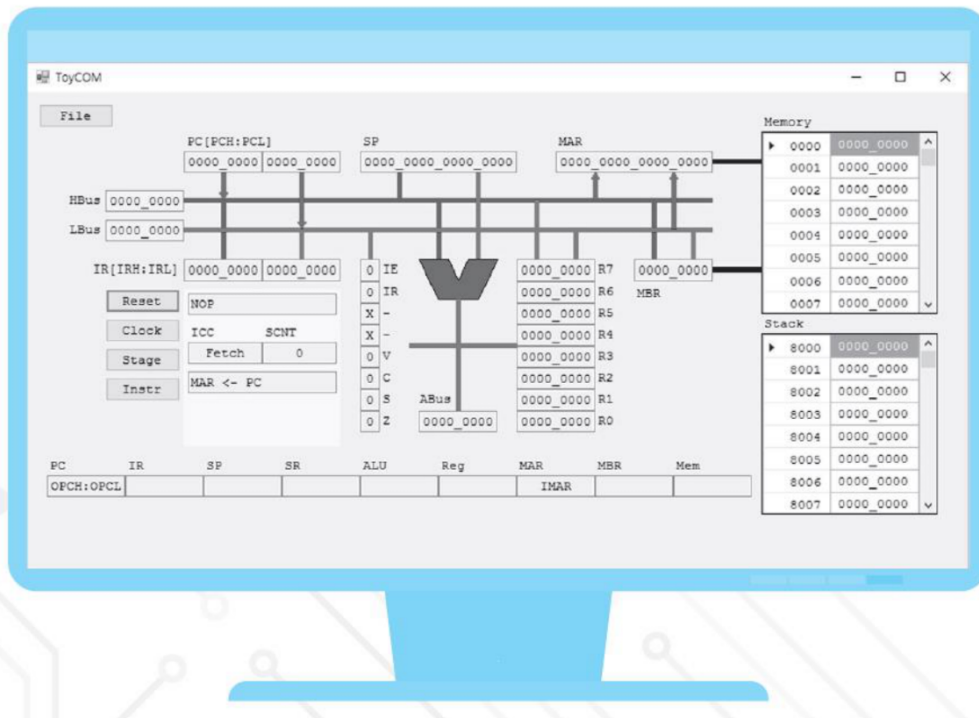


한눈에 보이는

개정 3 판

컴퓨터 구조



04. 중앙처리장치

4.1 중앙처리장치 구조

4.2 레지스터

4.3 인터럽트

4.4 명령어 사이클

4.5 요약

4.1 중앙처리장치 구조

중앙처리장치 기능 요약

- 프로그램 실행: 프로그램 = sequence of instructions
- 명령어 사이클: {인출 단계, 실행 단계}
- 인출 단계: $IR \leftarrow \text{Mem}(PC)$, $PC \leftarrow PC + \text{명령어(단어) 크기}$
- 실행 단계: 제어장치에서 명령어 해석하고, 처리장치에서 실행

내용

- 4.1.1 제어장치: 순서제어, 동작제어
- 4.1.2 처리장치: 레지스터 종류, 연산기 구조와 기능

중앙처리장치 구조

명령어 인출단계

1. $MAR \leftarrow PC$
2. $MBR \leftarrow Mem[MAR], PC \leftarrow PC + [명령어 길이]$
3. $IR \leftarrow MBR$

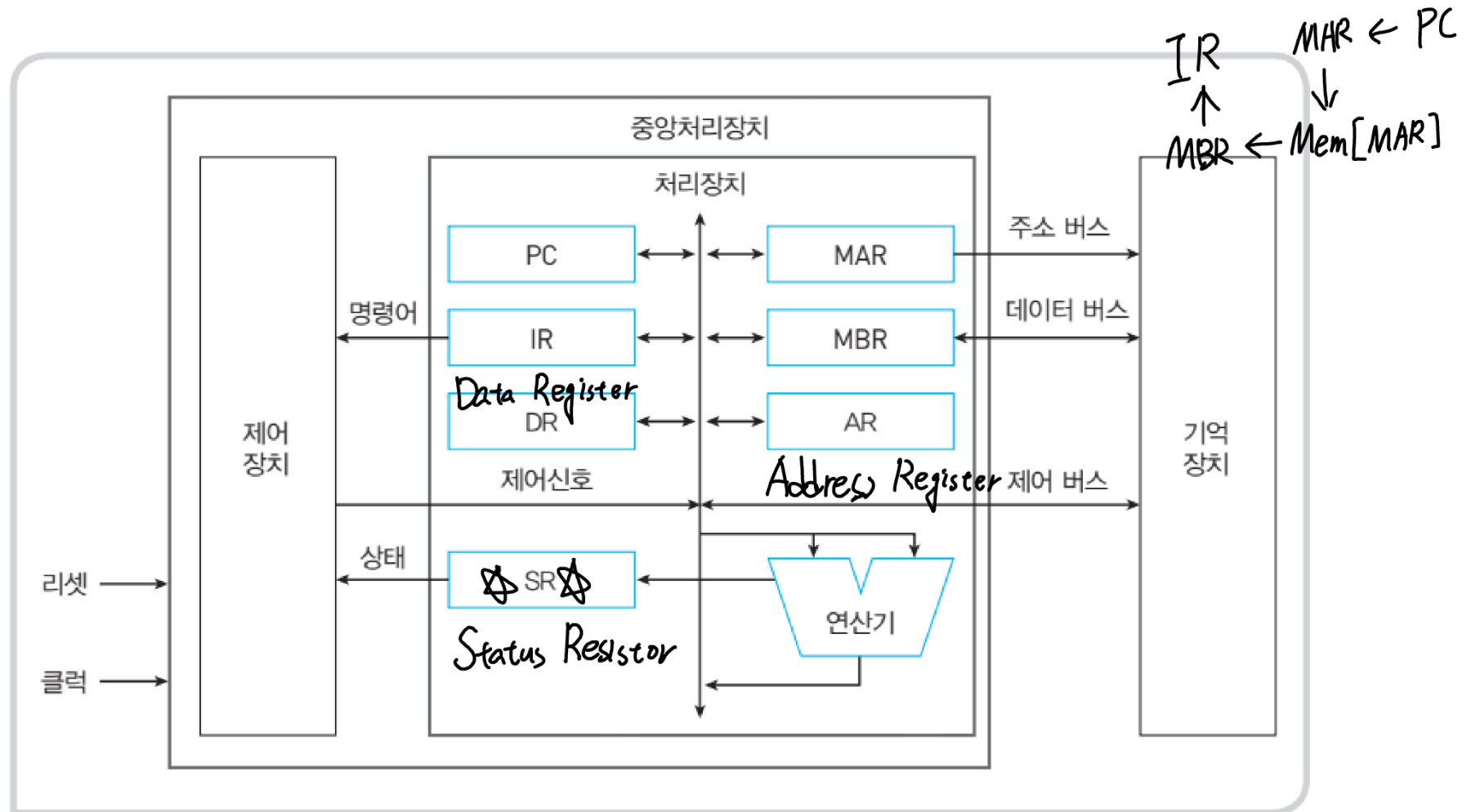


그림 4-1 중앙처리장치 구조

4.1.1 제어장치

제어장치 기능

- ↓
핵심
- 순서 제어(sequence control): 명령어 사이클 진행 명령어 차례대로 실행되도록 함
 - 명령어를 차례대로 실행한다.
 - 동작 제어(operation control): 제어신호 생성 연산기, 레지스터 등 제어
 - 각 명령어를 실행한다.

제어장치 입출력

- 입력
 - 명령어 레지스터(IR): 현재 실행하고 있는 명령어를 구별한다.
 - 상태 레지스터(SR): 조건 분기 처리를 위한 조건을 판단한다.
- 출력
 - 타이밍 신호: 명령어 사이클을 진행하기 위한 타이밍 신호를 생성한다.
 - 제어 신호: 컴퓨터의 구성 요소를 구동하는 제어 신호를 생성한다.

<순서제어>

명령어 사이클 진행 → 클럭 신호 입력받음 → 분기를 만나면 PC값을 바꿈

<동작제어>

명령어 레지스터 획득 → 현재 명령어가 무엇인지 → 명령어 실행하도록 제어신호 생성 → 분기를 만나면 SR에서 실행 여부 결정

4.1.2 처리장치

MUX, FID 등

레지스터 여러 비트를 저장하는 순차 논리회로 병행 블록 즉 기억장치의 액세스보다 빠름

- 중앙처리장치 내부 기억장치: 주소와 데이터 임시 저장
- 제어용 레지스터: PC, SR, MAR, MBR, IR : 프로그래머가 접근 못함
- 명령어 실행용 레지스터 : 프로그래머가 사용 가능
 - 데이터 레지스터 (DR), 주소 레지스터 (AR), 범용 레지스터 (GPR)

MAR
기억장치 주소 레지스터

MBR
기억장치 버퍼 레지스터

$GPR = DR + AR$

General Purpose Register

연산기: 산술, 논리연산 담당

- 데이터 처리 담당
- 제어신호는 연산기의 동작 결정
- 연산 후 상태를 상태 레지스터에 저장

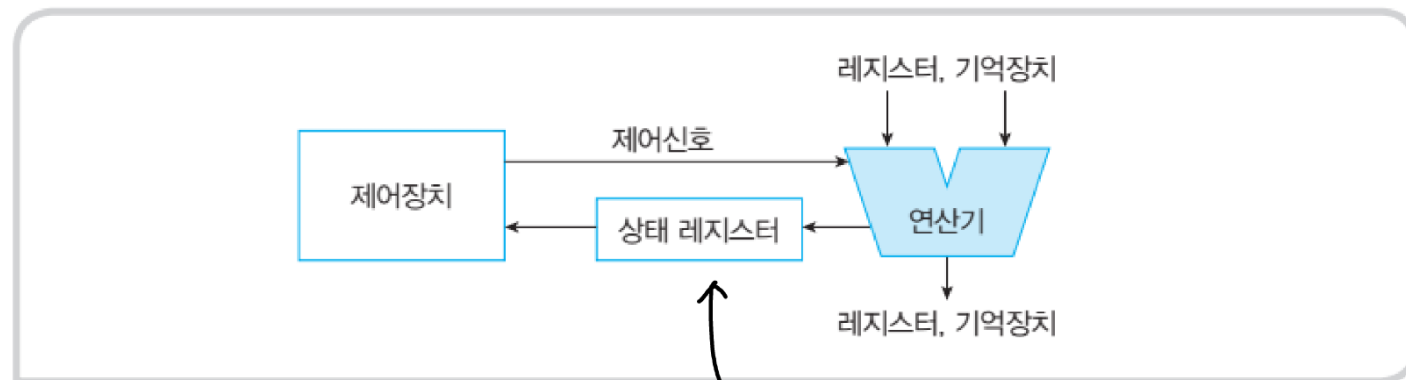


그림 4-2 연산기 주변 회로

SR은 연산기에 의해 바뀜

상태 레지스터 사용 예

Line 1: 비교 결과를 상태 레지스터에 저장

Line 2: 상태 레지스터를 참조하여 분기 여부 결정

BRG label 1
(갈라져라) (SR이 0보다 크면) (label 1으로)

1.	CMP	X, Y	// compare X and Y
2.	BRG	label1	// branch to label1 if greater than 0
3.	INC	X	// increment x
4. label1:	...		// branch to here

if 보다는 jump 느낌

X-Y 결과는 갖다 버리고 비교값만 남음
|| X, Y를 비교해라; 연산 결과가 0인지, >0인지 <0인지를 SR에 기록 (얼마나 크고 작으려는 관심X)

그림 4-3 조건분기 프로그램

= if (X <= Y) {
 X++;
}

4.2 레지스터

학습 목표

- 상태 레지스터 플래그의 기능을 설명할 수 있다.
- 시스템 스택 구조를 제시하고 스택 포인터를 활용 방법을 설명할 수 있다.
- 주소 레지스터의 기능과 용도를 설명할 수 있다.

내용

- 4.2.1 제어용 레지스터
- 4.2.2 상태 레지스터
- 4.2.3 명령어 실행용 레지스터
- 4.2.4 스택 포인터
- 4.2.5 베이스 레지스터
- 4.2.6 인덱스 레지스터
- 4.2.7 프로세서 레지스터

4.2.1 제어용 레지스터

프로세서가 프로그램을 진행하는데 사용하는 레지스터

프로그램 카운터(PC)

- 다음에 실행할 명령어가 저장된 기억장치 주소

명령어 레지스터(IR) *CPU에서 가장 튜링같은 보안*

- 현재 실행 중인 명령어

기억장치 주소 레지스터(MAR, Memory Address Register)

- 기억장치 주소 저장

(출력) 주소 버스에 연결

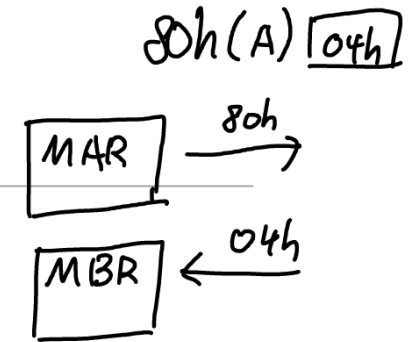
↑
명비호만 기억해심
↓

기억장치 버퍼 레지스터(MBR, Memory Buffer Register)

- 기억장치 데이터 중계
- 데이터 버스에 연결

기억장치 액세스

CPU: 기타 레지스터 많음

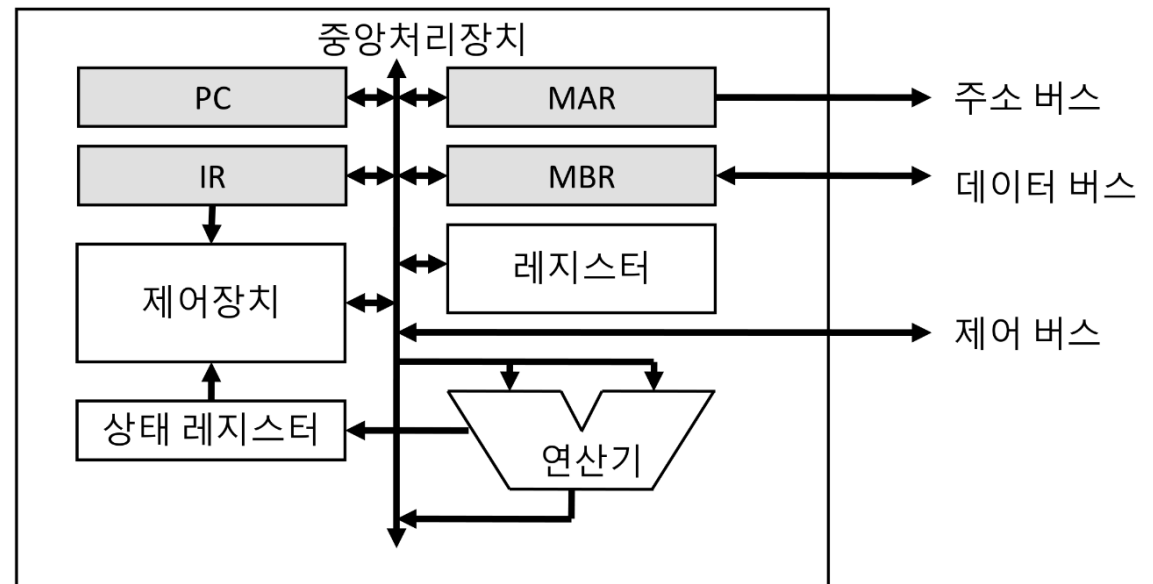


데이터 읽기

- 1) $MAR \leftarrow$ 주소 레지스터
- 2) $MBR \leftarrow Mem[MAR]$
- 3) 데이터 레지스터 $\leftarrow$ MBR

데이터 쓰기

- 1) $MAR \leftarrow$ 주소 레지스터
- 2) $MBR \leftarrow$ 데이터 레지스터
- 3) $Mem[MAR] \leftarrow MBR$



4.2.2 상태 레지스터

문제나옴

패리티 비트

→ 패리티는 2진수의 1의 개수 + 패리티 비트 해서
전체 1의 수를 홀수 아 짝수로 함
→ 일반적으로 짝수

(Even Parity System)

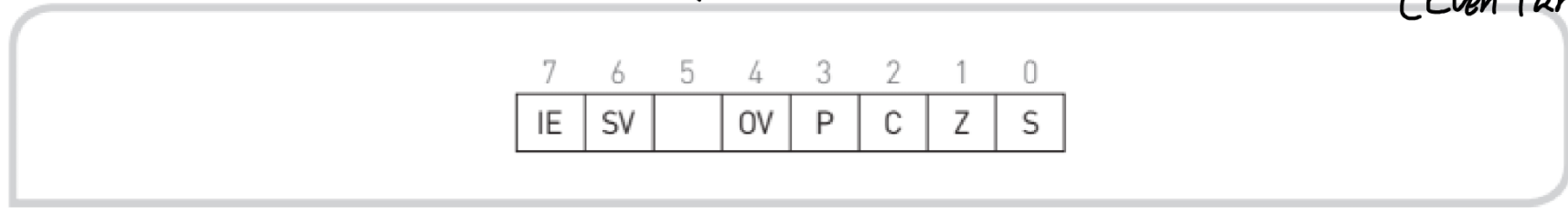


그림 4-4 8비트 상태 레지스터

Set : 1
reset : 0

→ SR의 각 비트를 flag라 부름

상태 레지스터 SR CPU는 프로그램 실행 상태를

플래그(flag)의 모임 변경할지 SR보고 판단

or bit
플래그 종류

- 조건 플래그(conditional flags)
 - 연산 결과 반영
- 제어 플래그(control flags)
 - 중앙처리장치 동작 제어

조건 플래그 1: 문수, 0: 양수

- 부호 (S, sign flag) 산술 연산 결과의 MSB 저장
- 제로 (Z, zero flag) 최근 연산 결과가 0이면 1, 아니면 0
- 자리올림수 (C, carry flag) 최근 연산이 2진수 덧셈이고, 자리올림이 발생할 때
- 패리티 (P, parity flag) 디지털공학 패리티체크 참고
- 오버플로우 (OV, overflow flag) 최근 연산 결과가 오버플로우 됐는지 표시

제어 플래그 CPU 동작 제어

인터럽트 (IE, Interrupt Enable flag)

운영체제 (SV, supervisor mode flag)

IE: 1일때 CPU는 외부 인터럽트 요청 인식 가능

SV: CPU가 어느 모드로 실행중인지

1: Kernel mode/supervisor mode : CPU가 전권을 가짐

0: user mode : 사용자가 쓴 프로그램이 권력

최근 연산 결과

10100011 : 101 4개

=> 패리티 비트: 0

10100111 : 101 5개

=> 패리티 비트: 1

조건 플래그

[예제 4-1] 덧셈 후 조건 플래그의 값을 구하라. (Even Parity)

$$\begin{array}{r} 0110_0101 \\ +0101_0100 \\ \hline 1011_1001 \end{array}$$

↙ MSB(최상위 비트)

- 1) S = 1 : sign flag (음수임)
- 2) Z = 0 : zero flag (0이 아님)
- 3) C = 0 : carry flag (최상위 비트의 carry만 따짐)
- 4) P = 1 : even parity
- 5) OV = 1 : overflow flag (양수 + 양수 = 음수 이므로 오버플로워)

4.2.3 명령어 실행용 레지스터

데이터 레지스터 (Data Register) *DR*

- 중앙처리장치가 처리하는 데이터 임시 저장

총 1개 컴퓨터: DR 1개; 누산기 (accumulator)

주소 레지스터 (Address Register) *AR = Pointer*

- 기억장치 주소 저장
- 중앙처리장치는 기억장치를 다양한 방법으로 액세스 한다.
- 스택 포인터, 베이스 레지스터, 인덱스 레지스터

범용 레지스터 (General Purpose Register) *프로그래머가 DR/AR로 사용 가능*

- 데이터 레지스터 + 주소 레지스터
- 통상 8 개 ~ 32 개 *= ToyCom의 R0 ~ R15*

GPR 중 일부는 Base Register, Index Register 지정

주소 레지스터

주소 레지스터

- 포인터(pointer): 주소 레지스터와 같은 의미
- 레지스터에 기억장치 주소를 저장하면, 명령어 길이가 짧아진다.

[예제 8-2] 기억장치 용량이 64K바이트고, 범용 레지스터가 32개다.

- 1) 명령어에 기억장치 주소를 표현하기 위한 비트 수는?
- 2) 명령어에 레지스터 중 하나를 표현하기 위한 비트 수는?



그림 4-5 포인터 개념

4.2.4 스택 포인터

컴퓨터는 메모리에서 일부를 System Stack 으로, 여기에 액세스하기 위해서 SP 사용

스택(stack) : LIFO (Last in - First out)

- Last-In-First-Out: 데이터를 쌓아 두었다가 하나씩 꺼내서 사용
- 한 곳에서 액세스: stack top $\leftarrow$ stack pointer

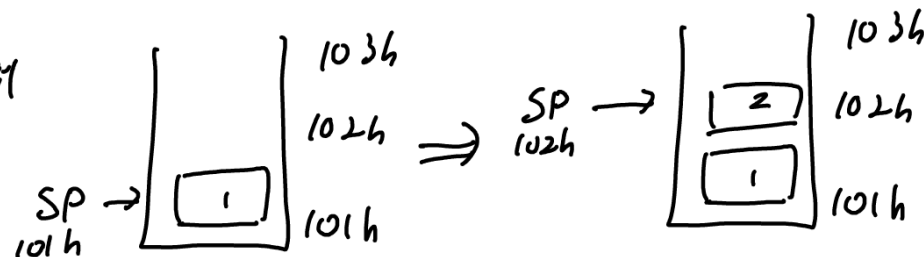
시스템 스택(system stack)

- 프로세서가 주기억장치의 일부를 스택 영역으로 활용
- 함수 호출시 리턴 주소 저장, 지역 변수 저장, 파라미터 전달

SP (Stack Pointer)

- 시스템 스택의 탑(top)을 가리키는 레지스터

마지막으로
추가된 데이터



스택 구조

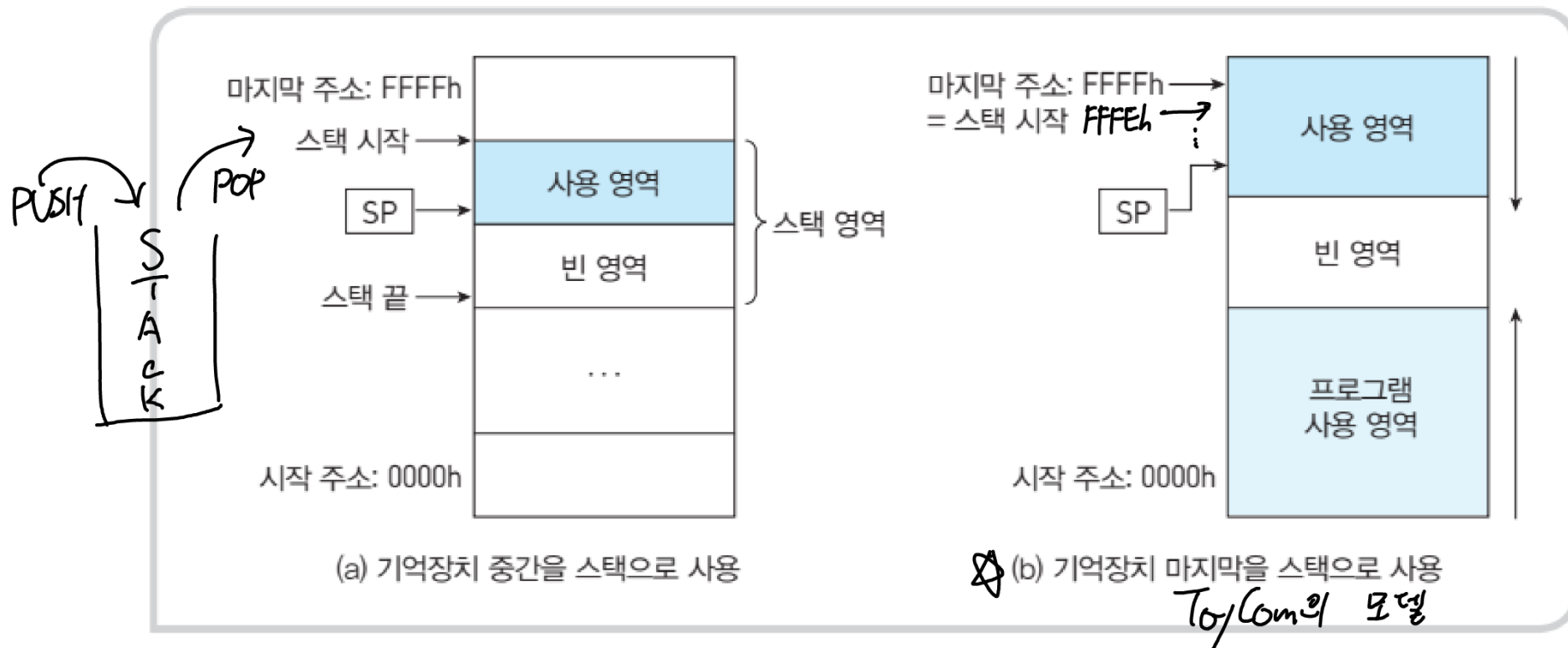


그림 4-6 스택 구조

그림 (b)

- SP가 가리키는 곳까지 데이터가 채워진 부분
- 데이터를 저장하면, SP 감소 / 데이터를 제거하면, SP 증가

주소 0번지부터 필요한 만큼 공간 사용
공간 없으면 오류

스택 동작

스택의 데이터 크기 = 레지스터 크기 = 단어 크기

PUSH 오퍼랜드:

$SP \leftarrow SP - [\text{단어 크기}]$

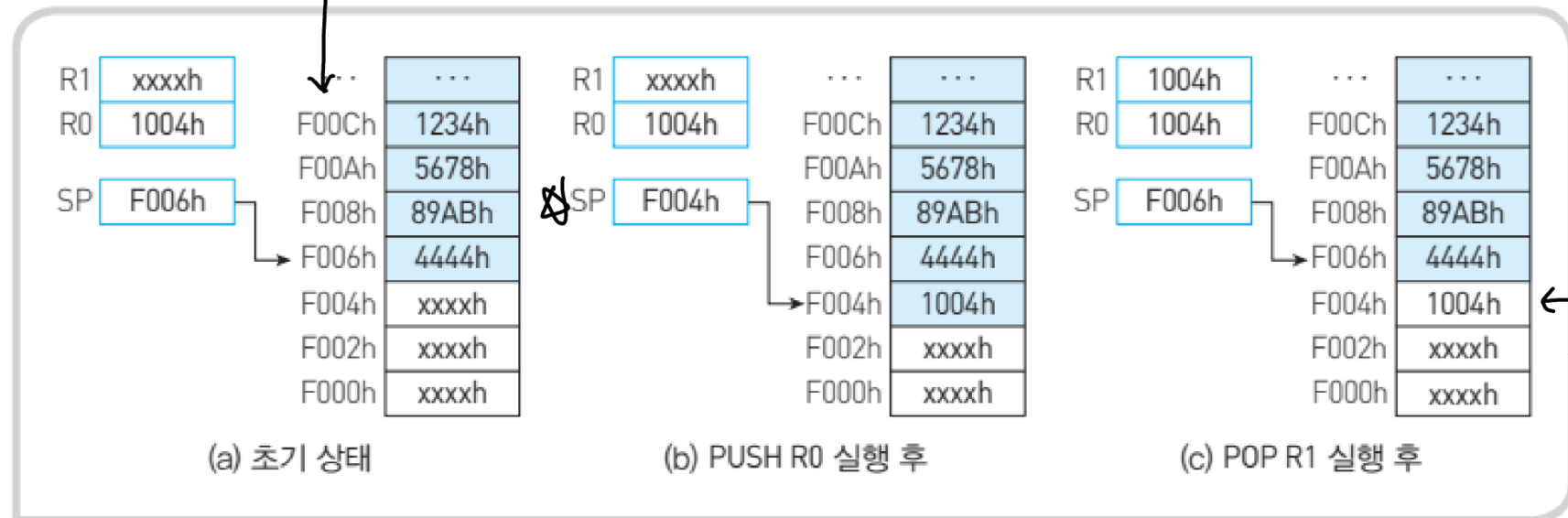
$\text{Mem}[SP] \leftarrow \text{오퍼랜드}$

메모리가 8비트, 레지스터가 16비트
=> 주소가 2개씩 줄어듦

POP 오퍼랜드:

오퍼랜드 $\leftarrow \text{Mem}[SP]$

$SP \leftarrow SP + [\text{단어 크기}]$



← 값은 안저장됨
쓰레기값으로
그냥 남아있는거
새로운 데이터
들어오면
이 값 지워지고
들어감
(스택으로 안봄)

☆ 그림 4-7 ☆ 스택 동작 ☆

스택 상태 변화

[예제 4-3] [그림 4-7(c)] 상태에서 다음 명령어 실행 후 스택의 변화

- 1) POP R0
- 2) POP R1
- 3) PUSH R0

POP: 데이터 → SP

PUSH: SP → 데이터

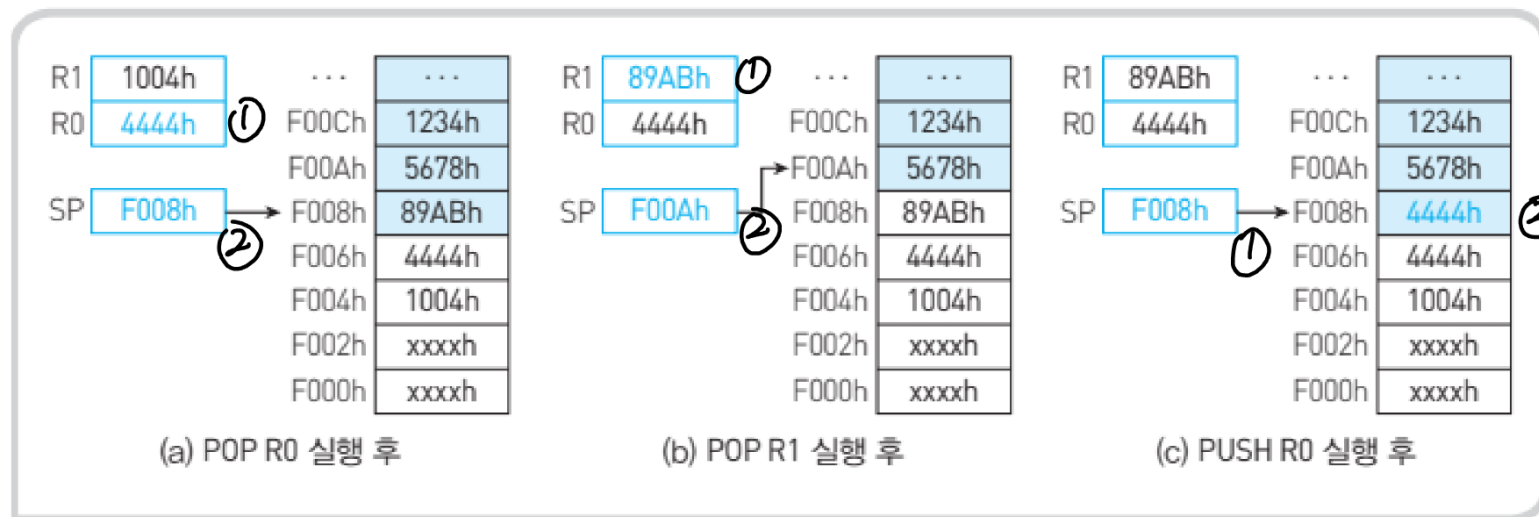


그림 4-8 예제 4-2의 명령어 실행 결과

스택 동작

[예제 4-4] MAR과 MBR을 고려한 스택 동작은?

PUSH 명령어:

$MBR \leftarrow \text{오퍼랜드}, SP \leftarrow SP - [\text{단어 크기}]$	// MBR 설정, SP 감소
$MAR \leftarrow SP$	// MAR 설정
$Mem[MAR] \leftarrow MBR$	// 기억장치에 저장

POP 명령어:

$MAR \leftarrow SP$	// MAR 설정
$MBR \leftarrow Mem[MAR]$	// 기억장치 읽기
$\text{오퍼랜드} \leftarrow MBR, SP \leftarrow SP + [\text{단어 크기}]$	// 레지스터 적재, SP 증가

스택 구현

표 4-1 스택 구현 방법에 따른 푸시와 팝 동작

SP 갱신 시기	SP 증가		SP 감소	
PUSH 명령어 실행 시 저장 전에 SP 갱신	PUSH	$SP \leftarrow SP + [\text{단어 크기}]$ $\text{Mem}[SP] \leftarrow \text{오퍼랜드}$	PUSH	$SP \leftarrow SP - [\text{단어 크기}]$ $\text{Mem}[SP] \leftarrow \text{오퍼랜드}$
	POP	$\text{오퍼랜드} \leftarrow \text{Mem}[SP]$ $SP \leftarrow SP - [\text{단어 크기}]$	POP	$\text{오퍼랜드} \leftarrow \text{Mem}[SP]$ $SP \leftarrow SP + [\text{단어 크기}]$
PUSH 명령어 실행 시 저장 후에 SP 갱신	PUSH	$\text{Mem}[SP] \leftarrow \text{오퍼랜드}$ $SP \leftarrow SP + [\text{단어 크기}]$	PUSH	$\text{Mem}[SP] \leftarrow \text{오퍼랜드}$ $SP \leftarrow SP - [\text{단어 크기}]$
	POP	$SP \leftarrow SP - [\text{단어 크기}]$ $\text{오퍼랜드} \leftarrow \text{Mem}[SP]$	POP	$SP \leftarrow SP + [\text{단어 크기}]$ $\text{오퍼랜드} \leftarrow \text{Mem}[SP]$

4.2.5 베이스 레지스터 스택 구현, 스택 동작, 베이스 레지스터는 수업 수업에서는 못들음ㅠㅠ

베이스 레지스터 (Base Register): 스택의 기준점 BR 건너뛰기

베이스 레지스터 활용 예제

```
void fn()
{
    ...
    function(x, y, z); // 파라미터 전달, 반환 주소 저장
    ...
}

void function(int param1, param2, param3)
{
    int local1, local2;
    ...
}
```


베이스 레지스터 활용

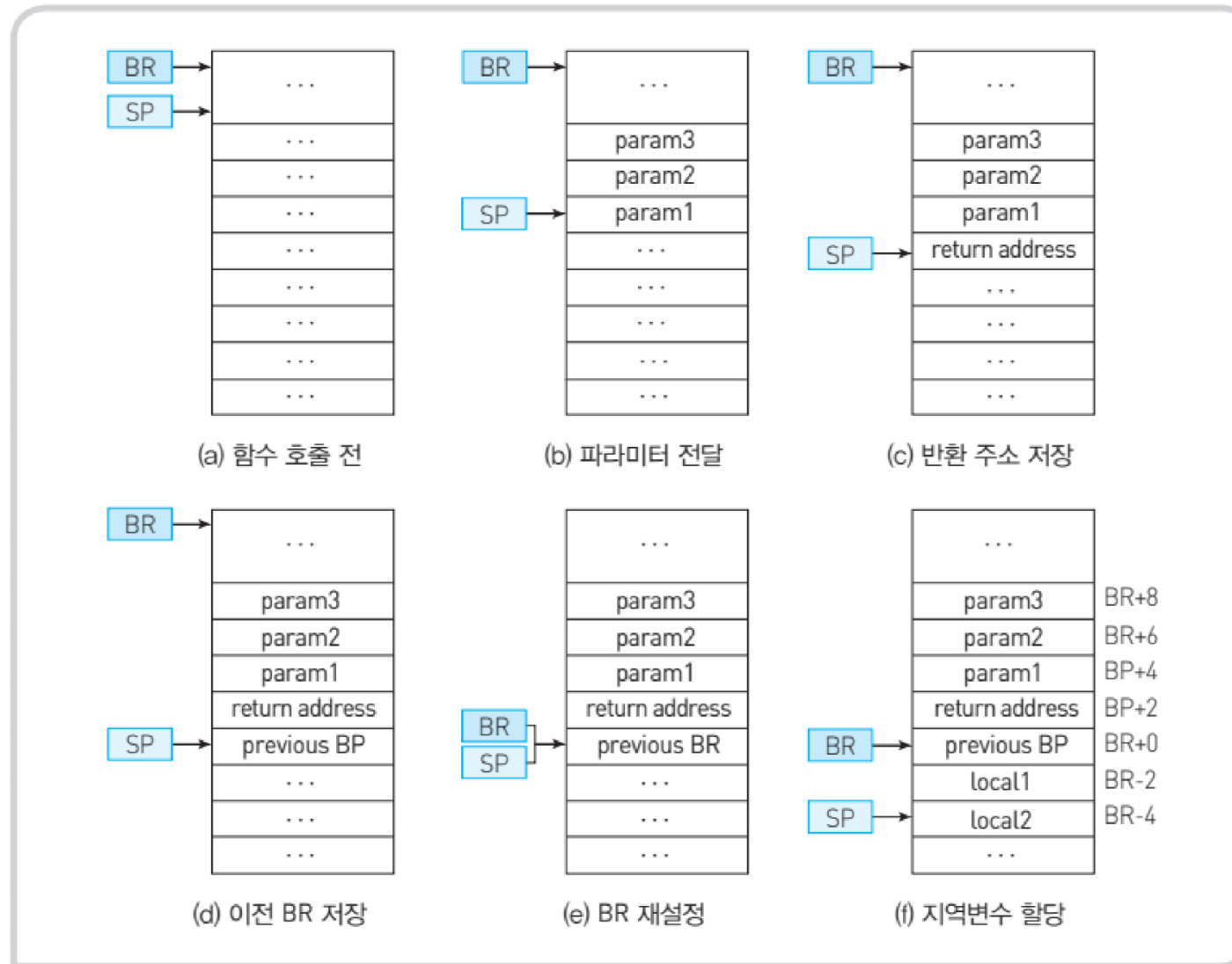


그림 4-9 함수 호출 과정

[예제 4-5]

함수 호출 조건

- 단어 크기는 기억장치 장소 한 개의 크기와 같고, 16비트다.
- 함수 호출하기 전 SP = F180h, BR = F188h이다.
- 함수를 function(4, 7, 10)으로 호출한다.
- 함수에서 리턴할 주소는 0420h이다.
- 함수의 지역 변수 값은 local1 = 15, local2 = 21이다.

함수 호출 후 SP, BR, 스택의 상태는?

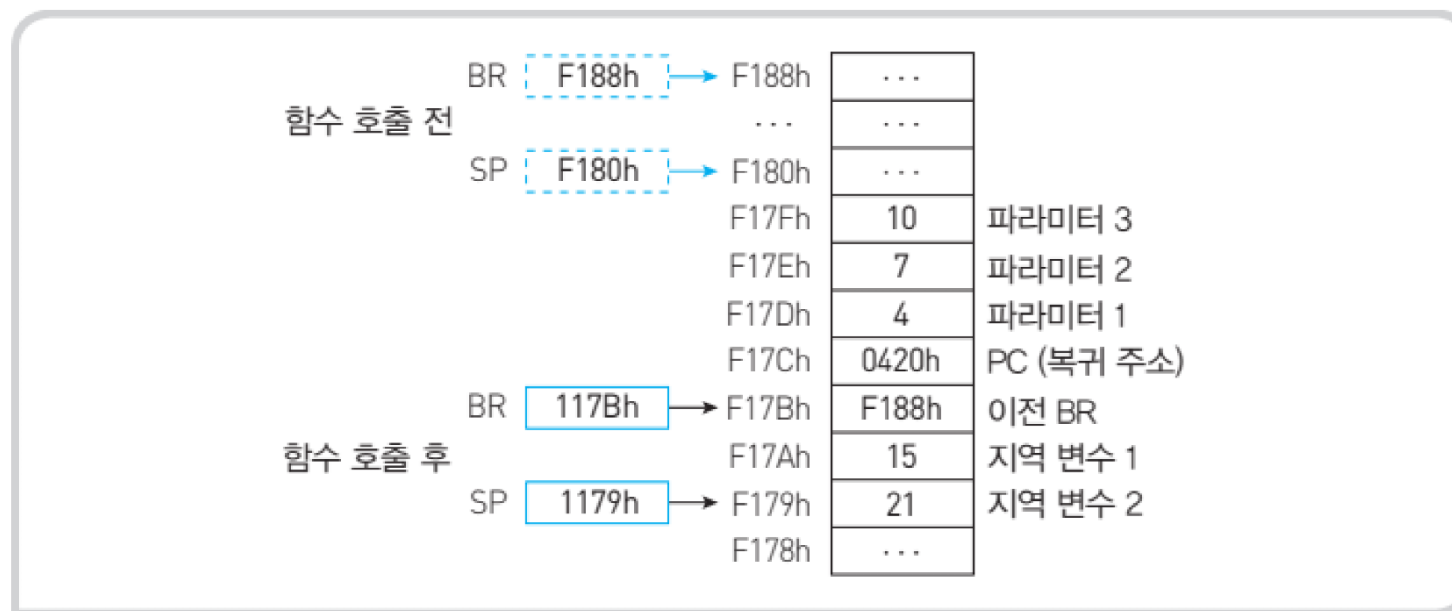


그림 4-10 예제 4-5의 함수 호출 후 상태

4.2.6 인덱스 레지스터 배열에 특화된 레지스터

인덱스 레지스터 (Index Register)

- 연속적으로 저장되어 있는 데이터(배열) 액세스
- 자동 인덱싱 (automatic indexing)

예

- `int array[5] = { 0, 1, 2, 3, 4 };` `// 1000h 번지부터 할당`

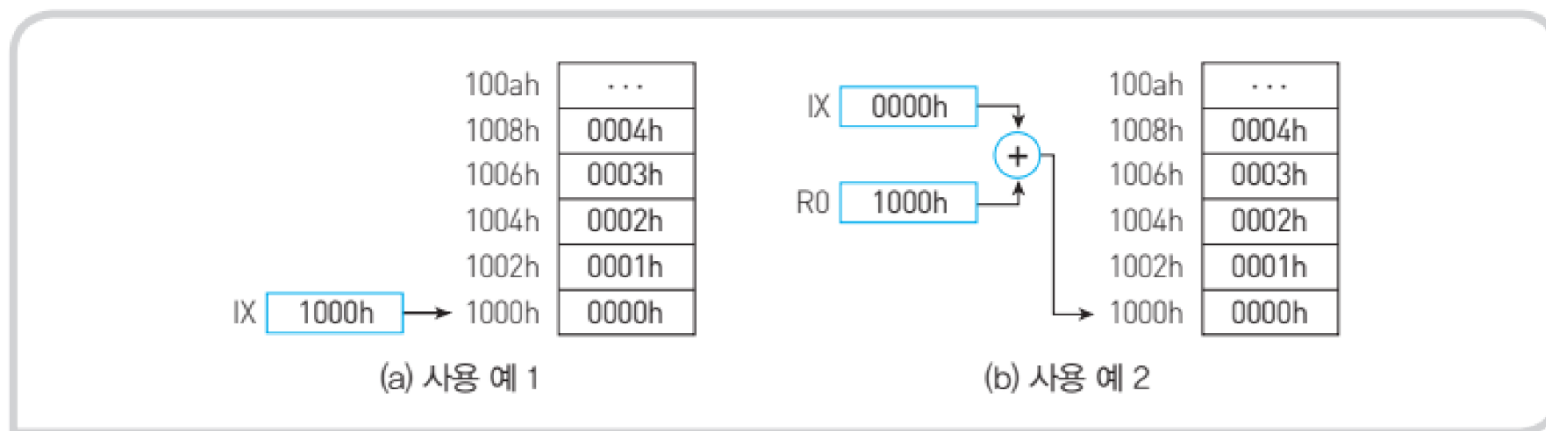


그림 4-11 인덱스 레지스터 사용 예

4.2.7 프로세서 레지스터 중요도 낮음

*인텔의
낮음 레지스터*

	31	16	15	8	7	0	
EAX					AH	AL	Accumulator
EBX					BH	BL	Base register
ECX					CH	CL	Count register
EDX					DH	DL	Data register
EBP					BP		Base Pointer
ESI					SI		Source index register
EDI					DI		Destination index register
	CS						Code segment
	DS						Data segment
	SS						Stack segment
	ES						Extra segment
	FS						Extended extra segment 1
	GS						Extended extra segment 2
EIP					IP		Instruction pointer
ESP					SP		Stack pointer
EFLAGS					FLAGS		Status register

(a) Intel Pentium 레지스터

16비트구조 유지 → 레지스터 복잡

	15	8	7	0	
PC					Program Counter
SP					Stack pointer
	SR				Status register
	R0				
	R1				
	R2				
	...	...			
	R25				
Register file	R26				X-register
	R27				
	R28				Y-register
	R29				
	R30				Z-register
	R31				

(b) AVR Atmega128 레지스터

그림 4-12 프로세서 레지스터

ARM 프로세서 레지스터 활용

문라오웬

표 4-2 ARM 프로세서의 레지스터

레지스터	APCS 이름	용도
R0-R3	a1-a4	아규먼트 전달, 임의 용도(a1은 정수 결과 반환)
R4-R8	v1-v5	레지스터 변수
R9	sb	스택 베이스(stack base)
R10	sl	스택 끝(stack limit)
R11	fp	프레임 포인터
R12	ip	새로운 스택 베이스(new stack base)
R13	sp	스택 포인터
R14	lr	링크 레지스터
R15	pc	프로그램 카운터

4.3 인터럽트 (CPU만 고려)

실제로는 OS나 프로그램에서도 발생

인터럽트

- 컴퓨터 내부 또는 외부에서 발생하는 갑작스러운 사건에 대응하는 기능

학습 목표

- 인터럽트 처리를 위하여 중앙처리장치가 제공하는 기능을 이해한다.
- 인터럽트 요청부터 서비스까지 처리 과정을 설명할 수 있다.

내용

- 4.3.1 인터럽트 개념
- 4.3.2 인터럽트 처리 과정
- 4.3.3 인터럽트 가능 플래그
- 4.3.4 인터럽트 서비스 루틴

4.3.1 인터럽트 개념

컴퓨터의 정상적인 프로그램 실행 방해

인터럽트(interrupt) - 방해하다.

- 중요
- 인터럽트 요청
 - 상태 저장
 - 인터럽트 서비스
 - 인터럽트 서비스에서 복귀

요청: 한 학생이 질문했다고 소리

저장: 전도를 어디까지 나갔나 기억

서비스: 질문 받음

복귀: 기억해둔 전도로 돌아감

인터럽트 종류

◦ 내부 인터럽트

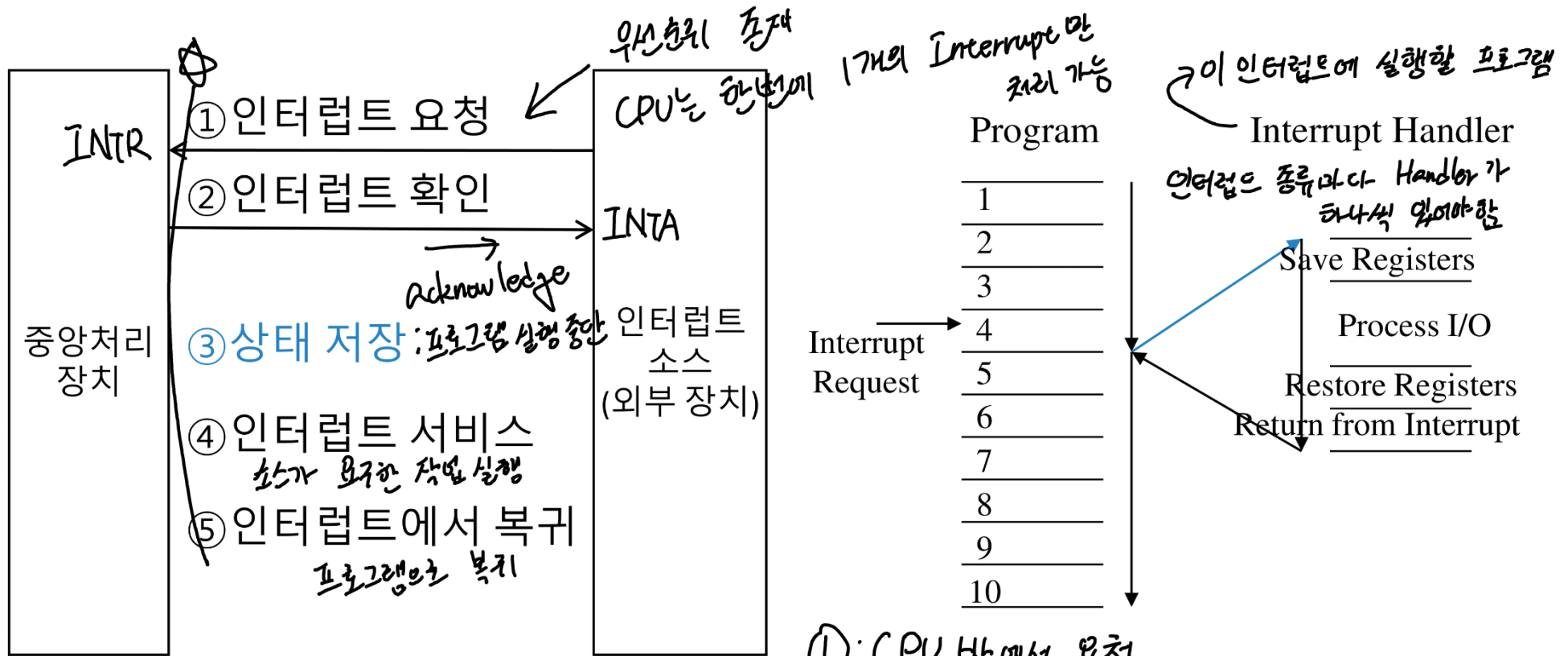
- 하드웨어 고장: 말 그대로 고장남
- 실행할 수 없는 명령어: 메모리에서 뺏은 게 해석 불가일때
- 명령어 실행 오류: 나누기 0: 명령어는 해석 가능, 명령어 자체가 실행안될때
- 사용 권한 위배: OS만 쓸수있는 자원 액세스 / 읽기 전용 파일에 쓰려고 할 때

◦ 외부 인터럽트

- 타이머 인터럽트: 이것 기반으로 시간 측정, 프로세스 실행시간 측정; 말 그대로 타이머 역할
- 입출력 인터럽트: 입출력 준대가 완료되었다고 CPU에게 알림
EX) 1초에 60번 인터럽트 → 60FPS
EX) 마우스 움직일 때마다 인터럽트 발생

4.3.2 인터럽트 처리 과정

INTR: 인터럽트 요청 신호
Interrupt Request
INTA: 인터럽트 확인 신호
Interrupt Acknowledge
→ 제어버스에 존재



①: CPU 밖에서 요청

②: CPU가 확인했다고 알려줌

ISR: Interrupt Service Routine = Interrupt Handler
→ 인터럽트 서비스 시 수행하는 프로그램

인터럽트 처리 기능

인터럽트 처리를 위해 필요한 사항

- 인터럽트 가능 플래그 (Interrupt Enable flag)
- 인터럽트 요청과 확인
- 인터럽트 단계
- 인터럽트 서비스 루틴 - 일반 프로그램

INTR

Interrupt Request

INTA

Interrupt Acknowledge

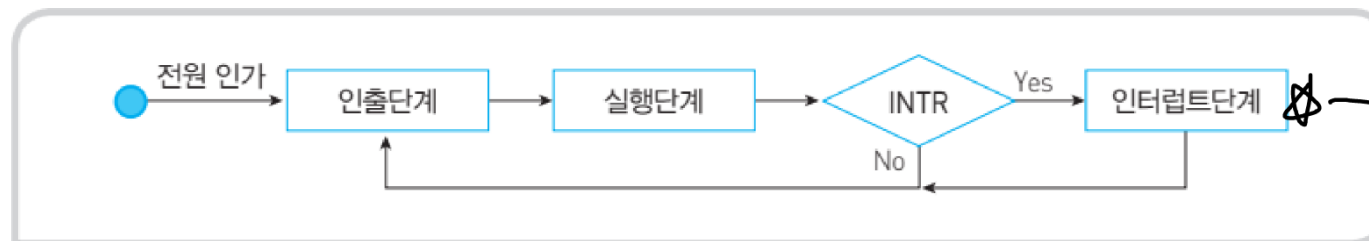


그림 4-13 인터럽트를 고려한 명령어 사이클

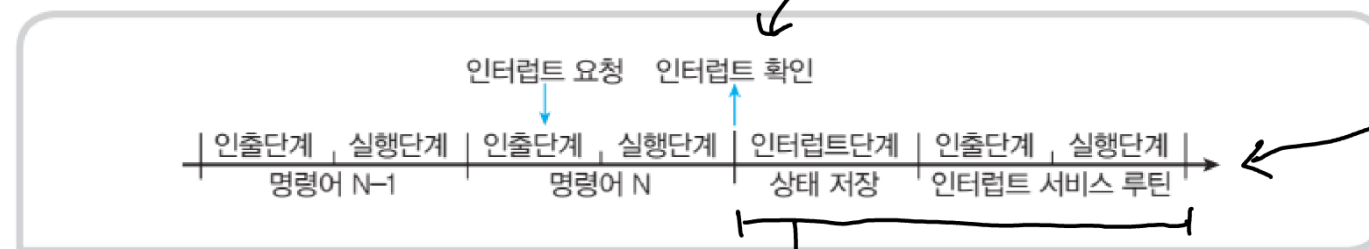


그림 4-14 인터럽트 요청과 확인

→ 스택에 저장, 인터럽트 소스 확인 후 해당 서비스 루틴의 주소를 PC에 적재
Push PC, Push SR
PC ← ISR

CPU가
프로그램으로
복귀할때

인터럽트 단계/리턴

필요한
최소정보 = Context
Interrupt와
관련된 Context: PC, SR

인터럽트 단계

- Context(PC와 SR)을 스택에 저장
- Interrupt source를 확인하고 ISR의 시작 주소 확인

```

SP ← SP - [단어 크기]           // PUSH PC
Mem[SP] ← PC
SP ← SP - [단어 크기]           // PUSH SR
Mem[SP] ← SR
PC ← ISR 시작 주소              // PC 갱신
    
```

PC → SR → 갱신

인터럽트 리턴

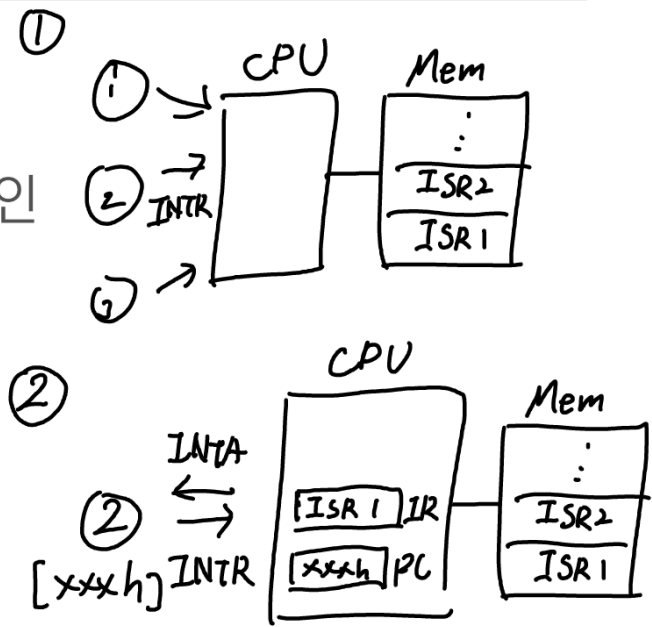
- 인터럽트 서비스 루틴의 마지막 실행 명령어 (Return from Interrupt)

```

SR ← Mem[SP]                    // POP SR
SP ← SP + [단어 크기]
PC ← Mem[SP]                    // POP PC
SP ← SP + [단어 크기]
    
```

SR → PC

인터럽트 단계



IRET

← Stack의 구조 때문 (LIFO)

ISR의 마지막 명령어는 IRET이어야 함

[예제 4-6] 인터럽트 단계 전/후 상태

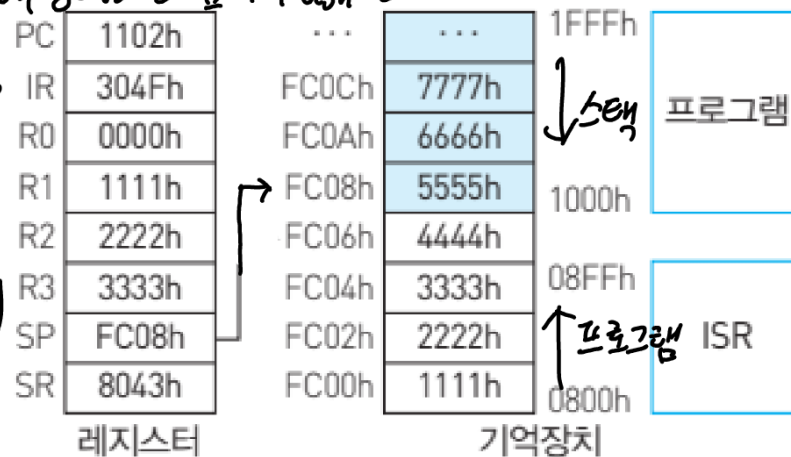
프로세서 상태

- 다음에 실행할 명령어 주소 PC=1102h
- 현재 실행 중인 명령어 코드 IR=304Fh
- 스택 탑: SP=FC08h
- ISR 시작 주소 = 0800h

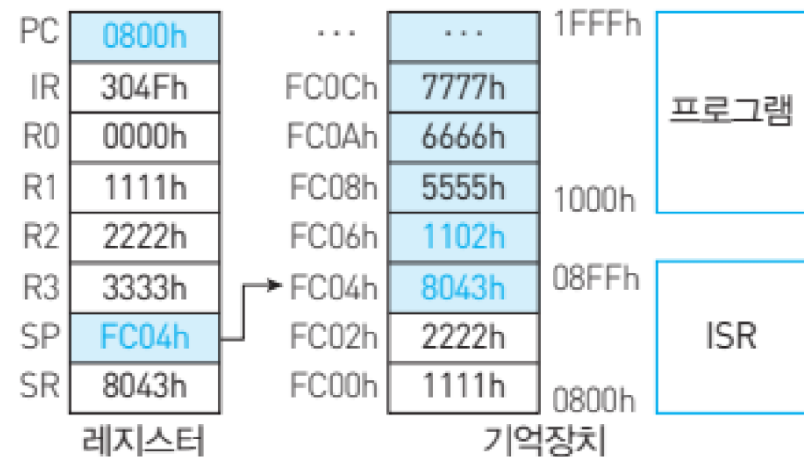
인터럽트 단계 실행 후

PUSH PC // SP ← FC06, Mem[SP] ← 1102h
 PUSH SR // SP ← FC04 Mem[SP] ← 8043h
 PC ← ISR 주소 // PC ← 0800

- ① SP가 6으로, 06에 1102 들어감; Push PC
 ② SP가 4로, 04에 8043 들어감; Push SR
 ③ PC에 800 넣음 : PC ← ISR
 (ISR의 시작 주소가 0800h 이기 때문)



(a) 인터럽트단계 실행 전



(b) 인터럽트단계 실행 후

그림 4-15 예제 4-6의 컴퓨터 상태

4.3.3 인터럽트 가능 플래그

컴퓨터의 상태 변화

- 시스템 리셋 후: 인터럽트 불가능 상태
- 초기화 과정 (부트로더)
 - 시스템 스택 설정
 - 인터럽트 서비스 루틴 적재
 - 인터럽트 벡터 설정
- 인터럽트 허용 상태로 전환

CPU는 Interrupt Vector Table 보고 무슨 ISR을 실행할지 결정

ISR도 프로그램

- 시스템 스택 설정
- ISR을 Mem에 적재

• Interrupt Vector Table 설정

→ 용어 서비스 가능
부트로더가 하는 일

Interrupt Vector: ISR의 시작 주소 (ISR의 시작주소 배열)

Interrupt Vector Table: Interrupt Vector를 모아놓은 영역

Boot Loader: 컴퓨터 전원 인가 후 처음으로 시작하는 프로그램

IE flag

- 상태 레지스터에 있는 제어 플래그 중 하나
- 중앙처리장치가 인터럽트를 허용할지 제어

	7	6	5	4	3	2	1	0
상태 레지스터	IE	SV		OV	P	C	Z	S

인터럽트 제어

IE 제어 명령어 ↗ IE를 1로

- STI (Set Interrupt Flag) : 인터럽트 허가
- CLI (Clear Interrupt Flag) : 인터럽트 거부

↘ IE를 0으로

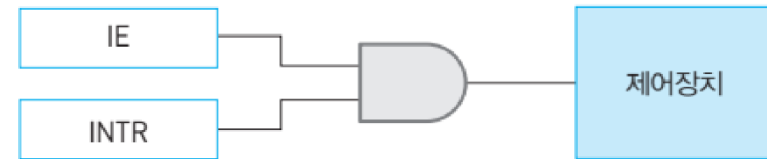


그림 4-16 인터럽트 가능 플래그

임계 영역 (critical section)

- 프로그램 중 실행이 중단되었다가 다시 시작되면 문제가 발생하는 영역
- IE 제어 명령어로 보호한다. ex) 현금 이체 프로그램중 인터럽트: 문제 발생 ⇒ Critical Section 사용

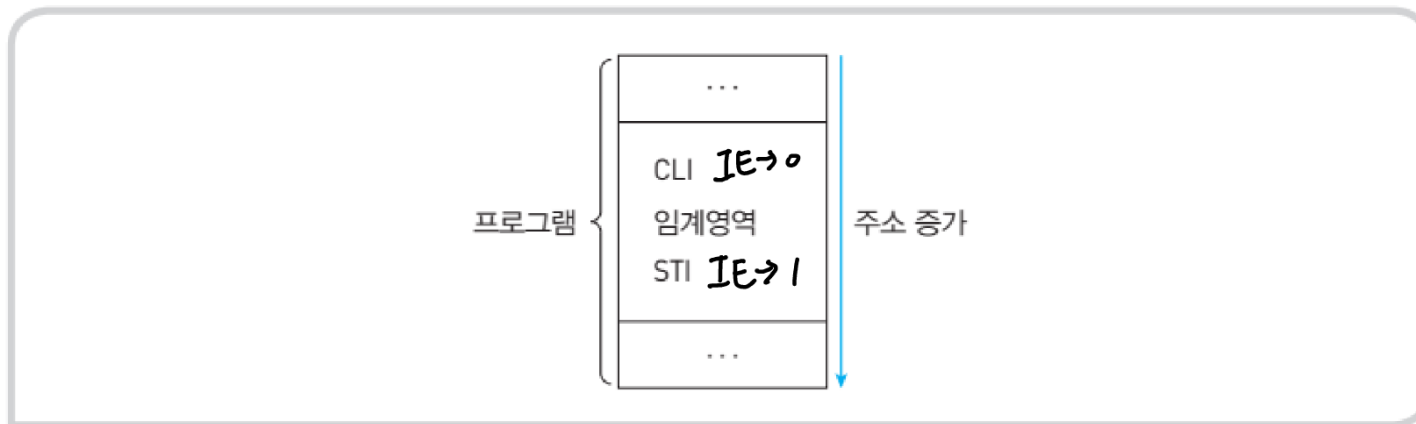


그림 4-17 임계영역을 포함하고 있는 프로그램

ISR은 자기가 쓸 레지스터 저장, 복구할 때 시스템 스택 사용

=> C++에서, 메소드를 만들 때 `int main()`에서 쓰는 변수와 메소드의 지역변수는 상호간섭X 즉, ISR은 시스템 스택을 사용해 프로그램과의 변수 충돌 방지

4.3.4 인터럽트 서비스 루틴

인터럽트 서비스 루틴(ISR, Interrupt Service Routine)

- 인터럽트 요청을 해결하는 프로그램
- Interrupt Handler

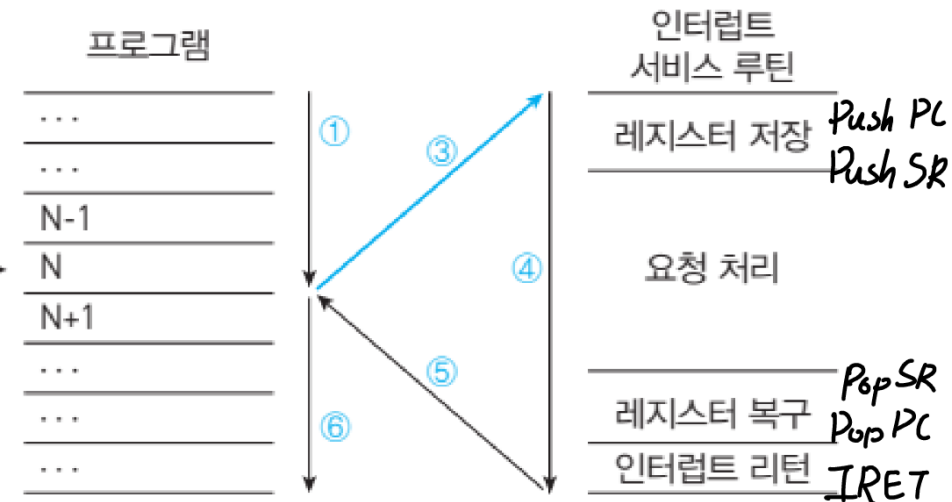
인터럽트 처리 순서

- ① 프로그램 실행 중
- ② 인터럽트 요청
 - 프로그램 실행과 비동기
- ③ 인터럽트 단계
 - 인터럽트 확인
 - 콘텍스트 저장 *Push PC, Push SR*
 - $PC \leftarrow \text{ISR 주소}$
- ④ 인터럽트 서비스 루틴 실행
- ⑤ 인터럽트 리턴
 - 콘텍스트 복구
- ⑥ 중단된 프로그램 다시 시작



인터럽트가 걸릴 지 모른다는 뜻

② 인터럽트 요청 →



인터럽트 처리 과정 및 인터럽트 서비스 루틴

인출단계, 인터럽트 단계에서 수행하는 일은 같음
실행 단계는 수행 일도 다르고, 수행 시간도 다름

4.4 명령어 사이클

명령어 사이클의 기본 구성: 인출 단계/실행 단계/인터럽트 단계

프로세서마다 명령어 사이클의 실행 단계를 세분화

- 해독 단계(decode stage): 명령어 코드를 해석
- 기록 단계(write stage): 명령어 실행 결과를 레지스터나 기억장치로 저장
- 간접 단계(indirect stage): 기억장치에서 데이터 인출

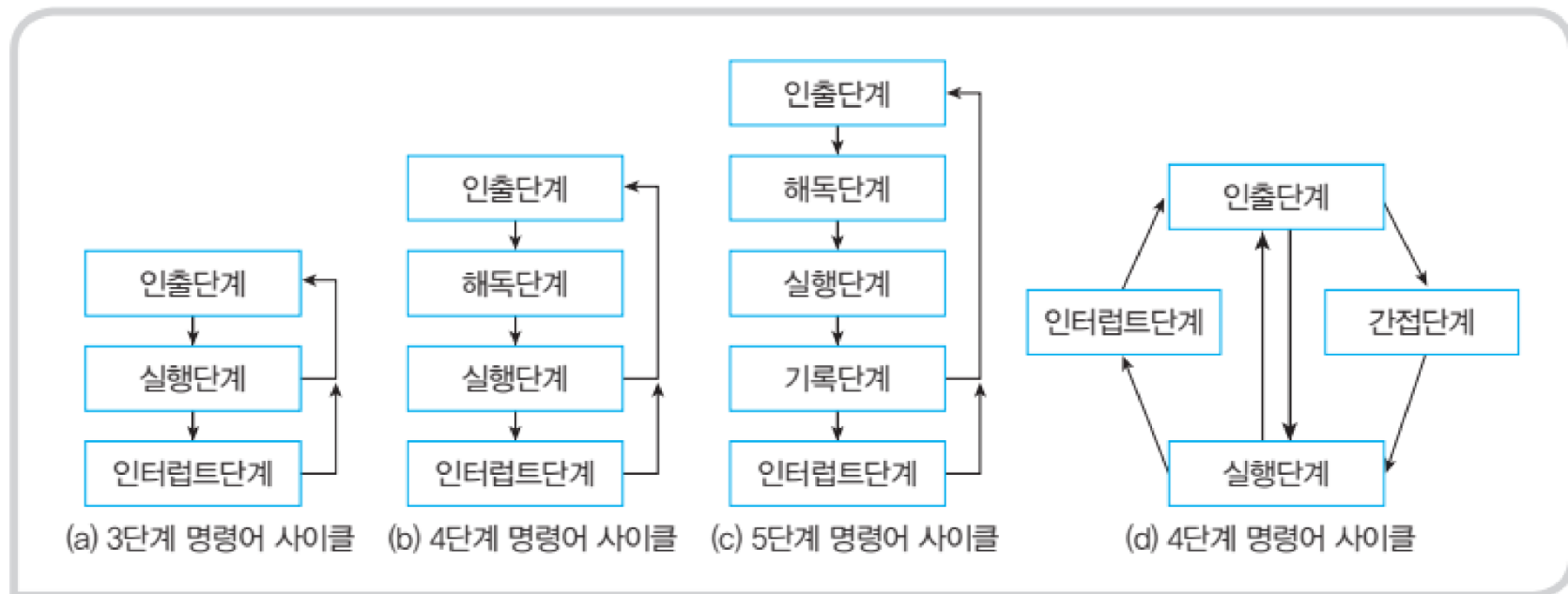


그림 4-19 명령어 사이클